

BITMAP Display

Pixel-driven bitmap

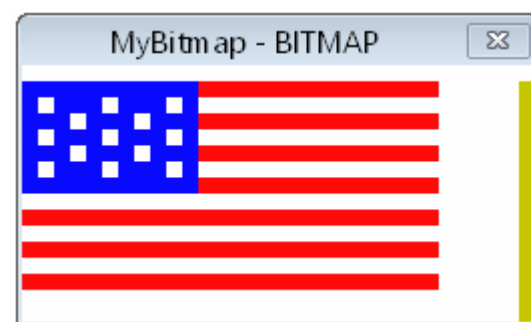
```
CON _clkfreq = 10_000_000

PUB go() | i

  debug(`bitmap MyBitmap SIZE 32 16 DOTSIZE 8 LUT2 LONGS_2BIT)
  debug(`MyBitmap TRACE 14 LUTCOLORS WHITE RED BLUE YELLOW 6)
  repeat
    debug(`MyBitmap `uhex_(flag[i++ & $1F]) `dly(100))

DAT

flag    long    %%333333333333333333
        long    %%001010101022222220
        long    %%001010101020202020
        long    %%001010101022222220
        long    %%0010101010220202020
        long    %%001010101022222220
        long    %%001010101020202020
        long    %%001010101022222220
        long    %%0010101010220202020
        long    %%001010101022222220
        long    %%001010101020202020
        long    %%001010101022222220
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%001010101010101010
        long    %%000000000000000000
        long    %%000000000000000000
        long    %%000000000000000000
        long    %%000000000000000000
        long    %%000000000000000000
```



BITMAP Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE x_pixels y_pixels	Set the number of pixels in the bitmap (1..2048 for both x and y).	256, 256
DOTSIZE x_and_y_size {y_size}	Set the size of the displayed pixels (1..256). 'DOTSIZE 16' makes each pixel 16x16 on the final display.	1, 1
lut1_to_rgb24	Set the color mode. See images below.	RGB24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with RGB24 colors. Use HEX_LONG_ARRAY_ to load.	default colors 0..7
TRACE 0_to_15	Set the pixel loading direction and whether to scroll after each line is filled. See animation below.	0
RATE pixels_per_update	Set the number of pixels before each display update. 'RATE -1' sets the rate to the bitmap size.	line size
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
BITMAP Feeding	Description	Default
lut1_to_rgb24	Change the color mode.	RGB24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load colors.	default colors 0..7
TRACE 0_to_15	Change the direction in which pixels are loaded into the bitmap. Sets the rate to the line size.	0
RATE pixels_per_update	Set the number of pixels before each display update. 'RATE -1' sets the rate to the bitmap size.	
SET x_position {y_position}	Set the current pixel-loading position. Cancels scroll mode by clearing bit 3 of TRACE.	
SCROLL x_scroll y_scroll	Scroll the bitmap by some number of pixels. Negative/positive values determine the direction, 0 = none.	
CLEAR	Clear the bitmap to zero-value pixels.	
UPDATE	Update the window with the current bitmap. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the bitmap at 1x scale.	
CLOSE	Close the window.	

TRACE modes

Rate is set to 1 so that each pixel can be seen as it's loaded.

0

1

2

3

4

5

6

7

8

9

10

11

12

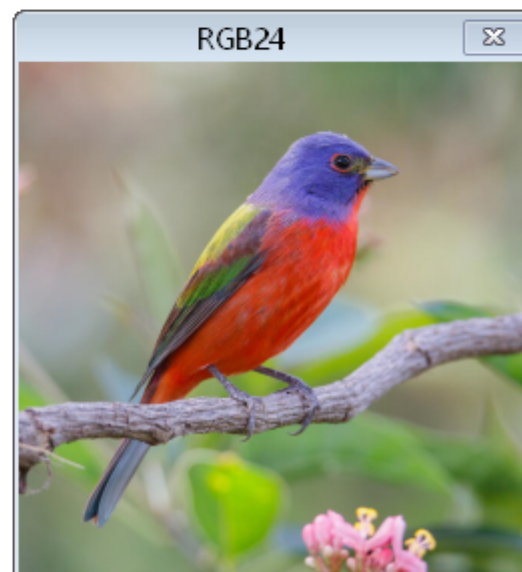
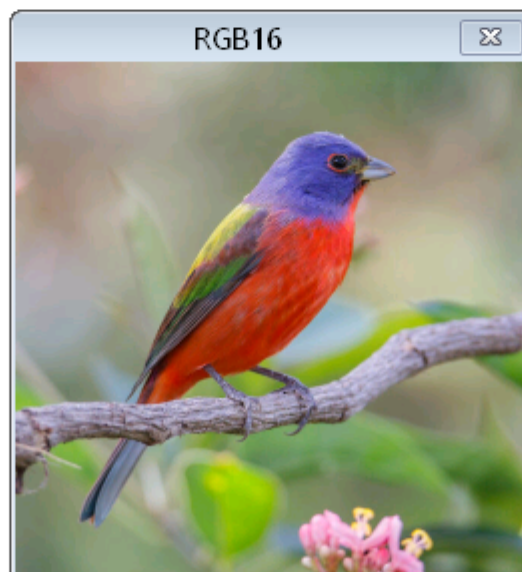
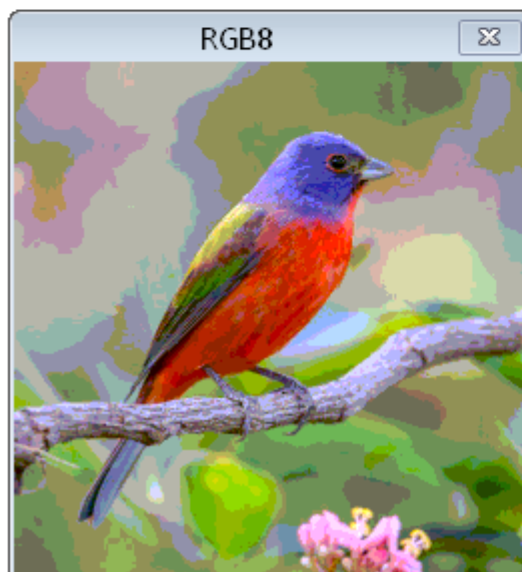
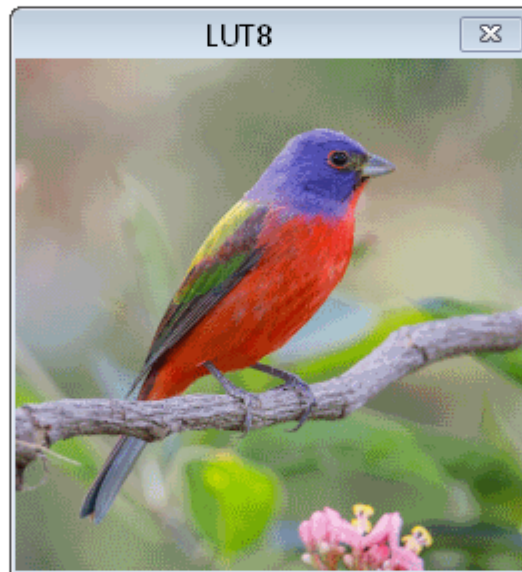
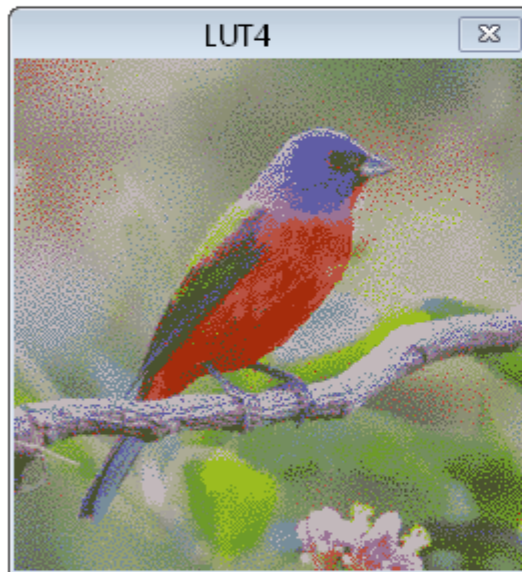
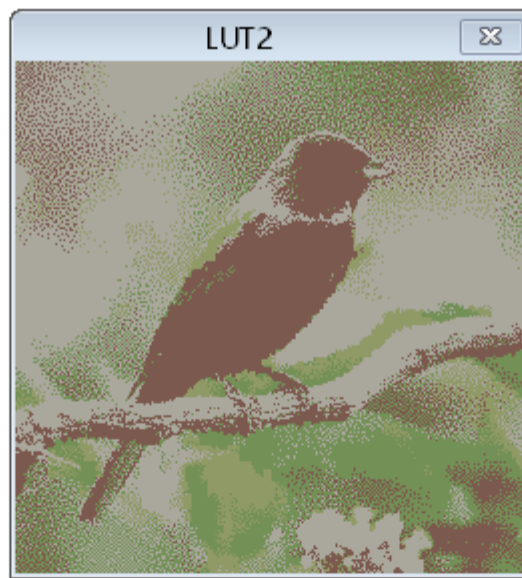
13

14

15

Color Mode	Bits/ Pixel	Description	Intention
LUT1	1	Pixel indexes LUT colors 0..1	Memory-efficient 2-color-palette graphics
LUT2	2	Pixel indexes LUT colors 0..3	Memory-efficient 4-color-palette graphics
LUT4	4	Pixel indexes LUT colors 0..15	Memory-efficient 16-color-palette graphics
LUT8	8	Pixel indexes LUT colors 0..255	Memory-efficient 256-color-palette graphics.
LUMA8	8	From black to color *	Instrumentation where luminance indicates level
LUMA8W	8	From white to color *	Instrumentation where saturation indicates level
LUMA8X	8	From black to color * to white	Instrumentation where luminance indicates level, peaking in white
HSV8	8	From black to color: %HHHHSSSS	16 hues with 16 luminance levels
HSV8W	8	From white to color: %HHHHSSSS	16 hues with 16 saturation levels, coming from white
HSV8X	8	From black to color to white: %HHHHSSSS	16 hues with 16 luminance levels, peaking in white
RGBI8	8	From black to color: %RGBIIIII	8 basic colors with 32 luminance levels
RGBI8W	8	From white to color: %RGBIIIII	8 basic colors with 32 saturation levels, coming from white
RGBI8X	8	From black to color to white: %RGBIIIII	8 basic colors with 32 luminance levels, peaking in white
RGB8	8	%RRRGGGBB	Byte-level RGB with 8 red, 8 green, and 4 blue levels
HSV16	16	From black to color: %HHHHHHHH_SSSSSSSS	256 hues with 256 luminance levels
HSV16W	16	From white to color: %HHHHHHHH_SSSSSSSS	256 hues with 256 saturation levels, coming from white
HSV16X	16	From black to color to white: %HHHHHHHH_SSSSSSSS	256 hues with 256 luminance levels, peaking in white
RGB16	16	%RRRRRGGG_GGGBBBBB	Word-level RGB with 32 red levels, 64 green levels, and 32 blue levels
RGB24	24	%RRRRRRRR_GGGGGGGG_BBBBBBBB	Full RGB with 256 levels for red, green, and blue

* Color is ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY.



```

CON _clkfreq = 100_000_000

PUB go() | i
  debug(`bitmap a title 'LUT1'   pos 100 100 trace 2 lut1 longs_1bit alt)
  debug(`bitmap b title 'LUT2'   pos 370 100 trace 2 lut2 longs_2bit alt)
  debug(`bitmap c title 'LUT4'   pos 100 395 trace 2 lut4 longs_4bit alt)
  debug(`bitmap d title 'LUT8'   pos 370 395 trace 2 lut8 longs_8bit)
  debug(`bitmap e title 'RGB8'   pos 100 690 trace 2 rgb8)
  debug(`bitmap f title 'RGB16'  pos 370 690 trace 2 rgb16)
  debug(`bitmap g title 'RGB24'  pos 640 690 trace 2 rgb24)
  waitms(1000)

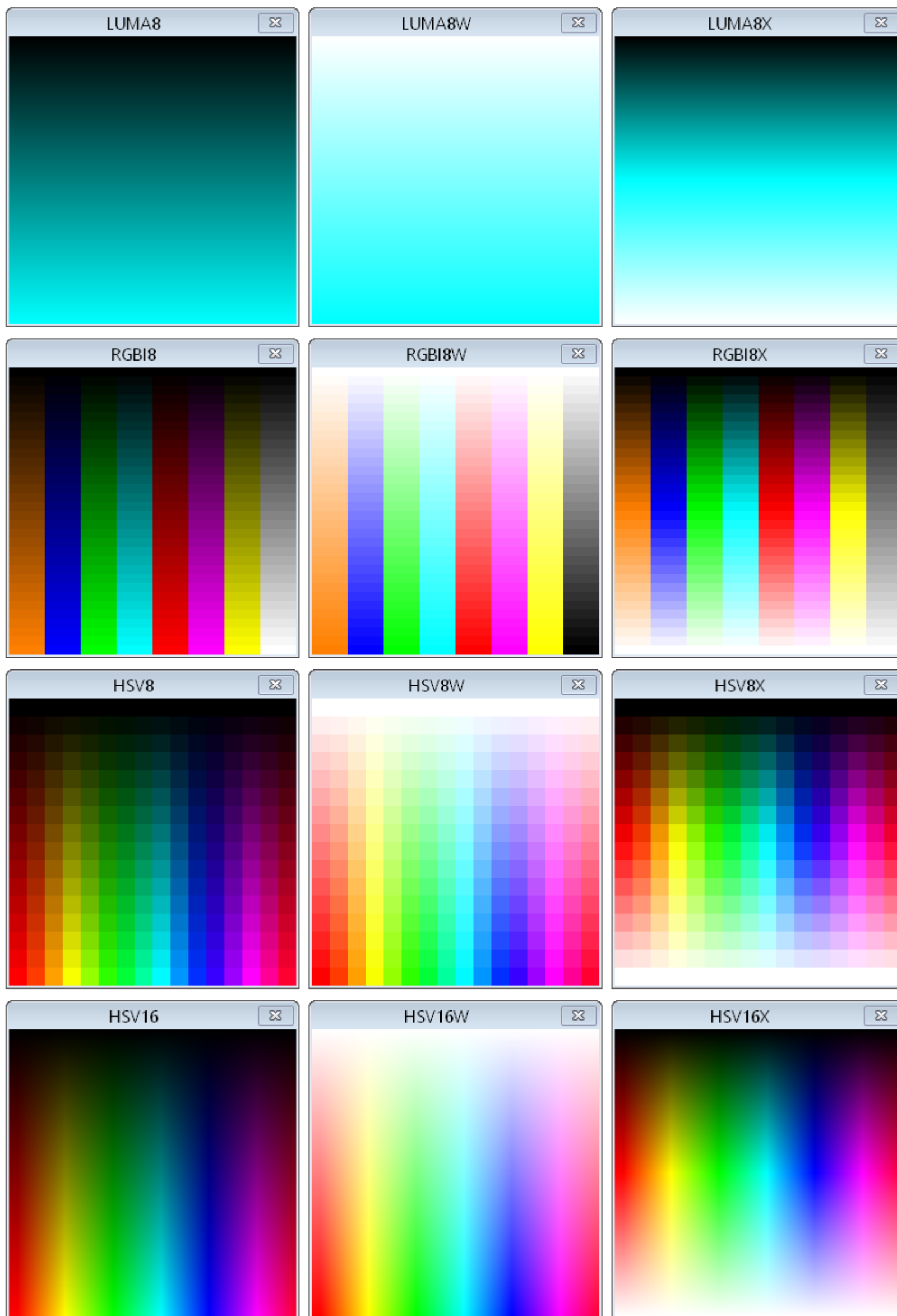
  showbmp("a", @image1, $8A, 2, $800) 'send LUT1 image
  showbmp("b", @image2, $36, 4, $1000) 'send LUT2 image
  showbmp("c", @image3, $8A, 16, $2000) 'send LUT4 image
  showbmp("d", @image4, $36, 256, $4000) 'send LUT8 image

  i := @image5 + $36 'send RGB8/RGB16/RGB24 images from the same 24-bpp file
  repeat $10000
    debug(`e `uhex_(byte[i+0] >> 6 + byte[i+1] >> 5 << 2 + byte[i+2] >> 5 << 5 ))
    debug(`f `uhex_(byte[i+0] >> 3 + byte[i+1] >> 2 << 5 + byte[i+2] >> 3 << 11))
    debug(`g `uhex_(byte[i+0] + byte[i+1] << 8 + byte[i+2] << 16 ))
    i += 3

PRI showbmp(letter, image_address, lut_offset, lut_size, image_long) | i
  image_address += lut_offset
  debug(`#(letter) lutcolors `uhex_long_array_(image_address, lut_size))
  image_address += lut_size << 2 - 4
  repeat image_long
    debug(`#(letter) `uhex_(long[image_address += 4]))

DAT
image1 file "bird_lut1.bmp"
image2 file "bird_lut2.bmp"
image3 file "bird_lut4.bmp"
image4 file "bird_lut8.bmp"
image5 file "bird_rgb24.bmp"

```



```

CON _clkfreq = 100_000_000

PUB go() | i
  debug(`bitmap a title 'LUMA8' pos 100 100 size 1 256 dotsize 256 1 luma8 cyan)
  debug(`bitmap b title 'LUMA8W' pos 370 100 size 1 256 dotsize 256 1 luma8w cyan)
  debug(`bitmap c title 'LUMA8X' pos 640 100 size 1 256 dotsize 256 1 luma8x cyan)
  debug(`bitmap d title 'RGBI8' pos 100 395 size 8 32 dotsize 32 8 trace 4 rgbi8)
  debug(`bitmap e title 'RGBI8W' pos 370 395 size 8 32 dotsize 32 8 trace 4 rgbi8w)
  debug(`bitmap f title 'RGBI8X' pos 640 395 size 8 32 dotsize 32 8 trace 4 rgbi8x)
  debug(`bitmap g title 'HSV8' pos 100 690 size 16 16 trace 4 dotsize 16 hsv8)
  debug(`bitmap h title 'HSV8W' pos 370 690 size 16 16 trace 4 dotsize 16 hsv8w)
  debug(`bitmap i title 'HSV8X' pos 640 690 size 16 16 trace 4 dotsize 16 hsv8x)
  debug(`bitmap j title 'HSV16' pos 100 985 size 256 256 trace 4 hsv16)
  debug(`bitmap k title 'HSV16W' pos 370 985 size 256 256 trace 4 hsv16w)
  debug(`bitmap l title 'HSV16X' pos 640 985 size 256 256 trace 4 hsv16x)
  waitms(1000)
  repeat i from 0 to 255
    debug(`a b c d e f g h i `uhex_(i))
  repeat i from 0 to 65535
    debug(`j k l `uhex_(i))

```


MIDI Display

MIDI keyboard for viewing note-on/off status with velocity

```
CON _clkfreq = 10_000_000

PUB go() | i

  debug(`midi MyMidi size 3 range 36 84)
  repeat
    repeat i from 36 to 84
      debug(`MyMidi $90 `(i, getrnd() & $7F))
      waitms(150)
      debug(`MyMidi $80 `(i, 0))
```

MIDI Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE keyboard_size	Set the size of the MIDI keyboard display (1..50).	4
RANGE first_key last_key	Set the first and last MIDI key numbers (0..127).	21, 108 (88 keys)
CHANNEL channel_number	Set the MIDI channel number to observe (0..15).	0
COLOR white_key black_key	Set the 'ON' colors for white and black keys. *	CYAN, MAGENTA
MIDI Feeding	Description	Default
byte	If \$90 + channel then NOTE_ON mode, else if \$80 + channel then NOTE_OFF mode. If NOTE_ON mode then receive a key (\$00..\$7F) and then its velocity (\$00..\$7F), update display. If NOTE_OFF mode then receive a key (\$00..\$7F) and then its velocity (\$00..\$7F), update display.	
CLEAR	Clear all notes.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

Here is a PASM program which receives MIDI serial on P16 and sends it to the MIDI display:

```
CON  _clkfreq      = 10_000_000
     rxpin         = 16

DAT  org

     asmclk

     debug  (`midi m size 2)

     wrpin  #%11111_0,#rxpin
     wxpin  ##(clkfreq_/31250) << 16 + 8-1, #rxpin
     drvl   #rxpin

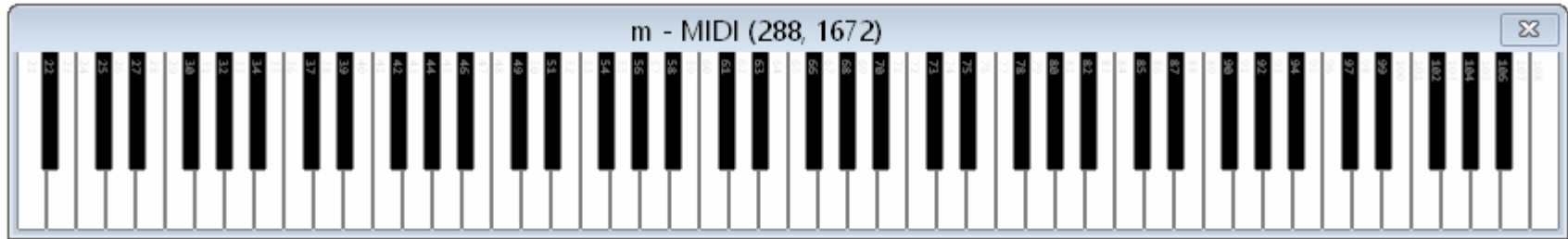
.wait  testp  #rxpin wc
     if_nc jmp  #.wait

     rdpin  x,#rxpin
     shr    x,#32-8

     debug  ("`m ", uhex_byte_(x))

     jmp    #.wait

x      res    1
```



Packed-Data Modes

Packed-data modes are used to efficiently convey sub-byte data types, by having the host side unpack them from bytes, words, or longs it receives. As well, bytes can be sent within words and longs, and words can be sent within longs for some efficiency improvement.

To establish packed-data operation, you must specify one of the modes listed below, followed by optional 'ALT' and 'SIGNED' keywords:

```
packed_mode {ALT} {SIGNED}
```

The **ALT** keyword will cause bits, double-bits, or nibbles, within each byte sent, to be reordered on the host side, within each byte. This simplifies cases where the raw data you are sending has its bitfields out-of-order with respect to the DEBUG display you are using. This is most-likely to be needed for bitmap data that was composed in standard formats.

The **SIGNED** keyword will cause all unpacked data values to be sign-extended on the host side.

Packed-Data Modes	Descriptions	Final Values	Final Values if SIGNED
LONGS_1BIT	Each value received is translated into 32 separate 1-bit values, starting from the LSB of the received value.	0..1	-1..0
LONGS_2BIT	Each value received is translated into 16 separate 2-bit values, starting from the LSBs of the received value.	0..3	-2..1
LONGS_4BIT	Each value received is translated into 8 separate 4-bit values, starting from the LSBs of the received value.	0..15	-8..7
LONGS_8BIT	Each value received is translated into 4 separate 8-bit values, starting from the LSBs of the received value.	0..255	-128..127
LONGS_16BIT	Each value received is translated into 2 separate 16-bit values, starting from the LSBs of the received value.	0..65,535	-32,768..32,767
WORDS_1BIT	Each value received is translated into 16 separate 1-bit values, starting from the LSB of the received value.	0..1	-1..0
WORDS_2BIT	Each value received is translated into 8 separate 2-bit values, starting from the LSBs of the received value.	0..3	-2..1
WORDS_4BIT	Each value received is translated into 4 separate 4-bit values, starting from the LSBs of the received value.	0..15	-8..7
WORDS_8BIT	Each value received is translated into 2 separate 8-bit values, starting from the LSBs of the received value.	0..255	-128..127
BYTES_1BIT	Each value received is translated into 8 separate 1-bit values, starting from the LSB of the received value.	0..1	-1..0
BYTES_2BIT	Each value received is translated into 4 separate 2-bit values, starting from the LSBs of the received value.	0..3	-2..1
BYTES_4BIT	Each value received is translated into 2 separate 4-bit values, starting from the LSBs of the received value.	0..15	-8..7

Built-In Symbols for Smart Pin Configuration

Smart Pin Symbol Value	Symbol Name	Details
A Input Polarity	(pick one)	
%0000_0000_000_00000000000000_00_00000_0	P_TRUE_A (default)	True A input
%1000_0000_000_00000000000000_00_00000_0	P_INVERT_A	Invert A input
A Input Selection	(pick one)	
%0000_0000_000_00000000000000_00_00000_0	P_LOCAL_A (default)	Select local pin for A input
%0001_0000_000_00000000000000_00_00000_0	P_PLUS1_A	Select pin+1 for A input
%0010_0000_000_00000000000000_00_00000_0	P_PLUS2_A	Select pin+2 for A input
%0011_0000_000_00000000000000_00_00000_0	P_PLUS3_A	Select pin+3 for A input
%0100_0000_000_00000000000000_00_00000_0	P_OUTBIT_A	Select OUT bit for A input
%0101_0000_000_00000000000000_00_00000_0	P_MINUS3_A	Select pin-3 for A input
%0110_0000_000_00000000000000_00_00000_0	P_MINUS2_A	Select pin-2 for A input
%0111_0000_000_00000000000000_00_00000_0	P_MINUS1_A	Select pin-1 for A input
B Input Polarity	(pick one)	
%0000_0000_000_00000000000000_00_00000_0	P_TRUE_B (default)	True B input
%0000_1000_000_00000000000000_00_00000_0	P_INVERT_B	Invert B input
B Input Selection	(pick one)	
%0000_0000_000_00000000000000_00_00000_0	P_LOCAL_B (default)	Select local pin for B input
%0000_0001_000_00000000000000_00_00000_0	P_PLUS1_B	Select pin+1 for B input
%0000_0010_000_00000000000000_00_00000_0	P_PLUS2_B	Select pin+2 for B input
%0000_0011_000_00000000000000_00_00000_0	P_PLUS3_B	Select pin+3 for B input
%0000_0100_000_00000000000000_00_00000_0	P_OUTBIT_B	Select OUT bit for B input
%0000_0101_000_00000000000000_00_00000_0	P_MINUS3_B	Select pin-3 for B input
%0000_0110_000_00000000000000_00_00000_0	P_MINUS2_B	Select pin-2 for B input
%0000_0111_000_00000000000000_00_00000_0	P_MINUS1_B	Select pin-1 for B input
A, B Input Logic	(pick one)	
%0000_0000_000_00000000000000_00_00000_0	P_PASS_AB (default)	Select A, B
%0000_0000_001_00000000000000_00_00000_0	P_AND_AB	Select A & B, B
%0000_0000_010_00000000000000_00_00000_0	P_OR_AB	Select A B, B
%0000_0000_011_00000000000000_00_00000_0	P_XOR_AB	Select A ^ B, B
%0000_0000_100_00000000000000_00_00000_0	P_FILT0_AB	Select FILT0 settings for A, B
%0000_0000_101_00000000000000_00_00000_0	P_FILT1_AB	Select FILT1 settings for A, B
%0000_0000_110_00000000000000_00_00000_0	P_FILT2_AB	Select FILT2 settings for A, B
%0000_0000_111_00000000000000_00_00000_0	P_FILT3_AB	Select FILT3 settings for A, B
Low-Level Pin Modes	(pick one)	
Logic/Schmitt/Comparator Input Modes		
%0000_0000_000_00000000000000_00_00000_0	P_LOGIC_A (default)	Logic level A → IN, output OUT
%0000_0000_000_00010000000000_00_00000_0	P_LOGIC_A_FB	Logic level A → IN, output feedback
%0000_0000_000_00100000000000_00_00000_0	P_LOGIC_B_FB	Logic level B → IN, output feedback
%0000_0000_000_00110000000000_00_00000_0	P_SCHMITT_A	Schmitt trigger A → IN, output OUT
%0000_0000_000_01000000000000_00_00000_0	P_SCHMITT_A_FB	Schmitt trigger A → IN, output feedback
%0000_0000_000_01010000000000_00_00000_0	P_SCHMITT_B_FB	Schmitt trigger B → IN, output feedback
%0000_0000_000_01100000000000_00_00000_0	P_COMPARE_AB	A > B → IN, output OUT
%0000_0000_000_01110000000000_00_00000_0	P_COMPARE_AB_FB	A > B → IN, output feedback
%xxxx_xxxx_xxx_xxxxSIOHHLLL_xx_xxxxx_x		Sync mode, IN/output polarity, high/low drive
ADC Input Modes		
%0000_0000_000_10000000000000_00_00000_0	P_ADC_GIO	ADC GIO → IN, output OUT
%0000_0000_000_10000100000000_00_00000_0	P_ADC_VIO	ADC VIO → IN, output OUT

%0000_0000_000_1000100000000_00_00000_0	P_ADC_FLOAT	ADC FLOAT → IN, output OUT
%0000_0000_000_1000110000000_00_00000_0	P_ADC_1X	ADC 1x → IN, output OUT
%0000_0000_000_1001000000000_00_00000_0	P_ADC_3X	ADC 3.16x → IN, output OUT
%0000_0000_000_1001010000000_00_00000_0	P_ADC_10X	ADC 10x → IN, output OUT
%0000_0000_000_1001100000000_00_00000_0	P_ADC_30X	ADC 31.6x → IN, output OUT
%0000_0000_000_1001110000000_00_00000_0	P_ADC_100X	ADC 100x → IN, output OUT
%xxxx_xxxx_xxx_xxxxxxOHHHLLL_xx_xxxxx_x		O = output polarity, HHH/LLL = high/low drive
DAC Output Modes		DIR enables output, OUT enables ADC
%0000_0000_000_1010000000000_00_00000_0	P_DAC_990R_3V	DAC 990Ω, 3.3V peak, ADC 1x → IN
%0000_0000_000_1010100000000_00_00000_0	P_DAC_600R_2V	DAC 600Ω, 2.0V peak, ADC 1x → IN
%0000_0000_000_1011000000000_00_00000_0	P_DAC_124R_3V	DAC 123.75Ω, 3.3V peak, ADC 1x → IN
%0000_0000_000_1011100000000_00_00000_0	P_DAC_75R_2V	DAC 75Ω, 2.0V peak, ADC 1x → IN
%xxxx_xxxx_xxx_xxxxxDDDDDDDD_xx_xxxxx_x		DDDDDDDD = 8-bit DAC value
Level-Comparison Modes		DIR enables output (1.5kΩ drive)
%0000_0000_000_1100000000000_00_00000_0	P_LEVEL_A	A > Level → IN, output OUT
%0000_0000_000_1101000000000_00_00000_0	P_LEVEL_A_FBN	A > Level → IN, output negative feedback
%0000_0000_000_1110000000000_00_00000_0	P_LEVEL_B_FBP	B > Level → IN, output positive feedback
%0000_0000_000_1111000000000_00_00000_0	P_LEVEL_B_FBN	B > Level → IN, output negative feedback
%xxxx_xxxx_xxx_xxxxSLLLLLLLL_xx_xxxxx_x		S = Synchronous, LLLLLLLL = 8-bit Level
Low-Level Pin Sub-Modes		
Sync Mode	(pick one)	(for Logic/Schmitt/Comparator/Level modes)
%xxxx_xxxx_xxx_xxxxSxxxxxxxx_xx_xxxxx_x		Sync mode bit
%0000_0000_000_0000000000000_00_00000_0	P_ASYNC_IO (default)	Select asynchronous I/O
%0000_0000_000_0000100000000_00_00000_0	P_SYNC_IO	Select synchronous I/O
IN Polarity	(pick one)	(for Logic/Schmitt/Comparator modes)
%xxxx_xxxx_xxx_xxxxIxxxxxxxx_xx_xxxxx_x		IN polarity bit
%0000_0000_000_0000000000000_00_00000_0	P_TRUE_IN (default)	True IN bit
%0000_0000_000_0000010000000_00_00000_0	P_INVERT_IN	Invert IN bit
Output Polarity	(pick one)	(for Logic/Schmitt/Comparator/ADC modes)
%xxxx_xxxx_xxx_xxxxxxOxxxxxx_xx_xxxxx_x		Output polarity bit
%0000_0000_000_0000000000000_00_00000_0	P_TRUE_OUTPUT (default)	Select true output
%0000_0000_000_0000001000000_00_00000_0	P_INVERT_OUTPUT	Select inverted output
Drive-High Strength	(pick one)	(for Logic/Schmitt/Comparator/ADC modes)
%xxxx_xxxx_xxx_xxxxxxxHHHxx_xx_xxxxx_x		Drive-high selector bits
%0000_0000_000_0000000000000_00_00000_0	P_HIGH_FAST (default)	Drive high fast (30mA)
%0000_0000_000_0000000001000_00_00000_0	P_HIGH_1K5	Drive high 1.5kΩ
%0000_0000_000_0000000010000_00_00000_0	P_HIGH_15K	Drive high 15kΩ
%0000_0000_000_0000000011000_00_00000_0	P_HIGH_150K	Drive high 150kΩ
%0000_0000_000_0000000100000_00_00000_0	P_HIGH_1MA	Drive high 1mA
%0000_0000_000_0000000101000_00_00000_0	P_HIGH_100UA	Drive high 100μA
%0000_0000_000_0000000110000_00_00000_0	P_HIGH_10UA	Drive high 10μA
%0000_0000_000_0000000111000_00_00000_0	P_HIGH_FLOAT	Float high
Drive-Low Strength	(pick one)	(for Logic/Schmitt/Comparator/ADC modes)
%xxxx_xxxx_xxx_xxxxxxxxxxLLL_xx_xxxxx_x		Drive-low selector bits
%0000_0000_000_0000000000000_00_00000_0	P_LOW_FAST (default)	Drive low fast (30mA)
%0000_0000_000_0000000000001_00_00000_0	P_LOW_1K5	Drive low 1.5kΩ
%0000_0000_000_0000000000010_00_00000_0	P_LOW_15K	Drive low 15kΩ
%0000_0000_000_0000000000011_00_00000_0	P_LOW_150K	Drive low 150kΩ
%0000_0000_000_000000000100_00_00000_0	P_LOW_1MA	Drive low 1mA
%0000_0000_000_000000000101_00_00000_0	P_LOW_100UA	Drive low 100μA
%0000_0000_000_000000000110_00_00000_0	P_LOW_10UA	Drive low 10μA

%0000_0000_000_0000000000111_00_00000_0	P_LOW_FLOAT	Float low
DIR/OUT Control	(pick one)	
%0000_0000_000_0000000000000_00_00000_0	P_TT_00 (default)	TT = %00
%0000_0000_000_0000000000000_01_00000_0	P_TT_01	TT = %01
%0000_0000_000_0000000000000_10_00000_0	P_TT_10	TT = %10
%0000_0000_000_0000000000000_11_00000_0	P_TT_11	TT = %11
%0000_0000_000_0000000000000_01_00000_0	P_OE	Enable output in smart pin mode
%0000_0000_000_0000000000000_01_00000_0	P_CHANNEL	Enable DAC channel in non-smart pin DAC mode
%0000_0000_000_0000000000000_10_00000_0	P_BITDAC	Enable BITDAC for non-smart pin DAC mode
Smart Pin Modes	(pick one)	
%0000_0000_000_0000000000000_00_00000_0	P_NORMAL (default)	Normal mode (not smart pin mode)
%0000_0000_000_0000000000000_00_00001_0	P_REPOSITORY	Long repository (non-DAC mode)
%0000_0000_000_0000000000000_00_00001_0	P_DAC_NOISE	DAC Noise (DAC mode)
%0000_0000_000_0000000000000_00_00010_0	P_DAC_DITHER_RND	DAC 16-bit random dither (DAC mode)
%0000_0000_000_0000000000000_00_00011_0	P_DAC_DITHER_PWM	DAC 16-bit PWM dither (DAC mode)
%0000_0000_000_0000000000000_00_00100_0	P_PULSE	Pulse/cycle output
%0000_0000_000_0000000000000_00_00101_0	P_TRANSITION	Transition output
%0000_0000_000_0000000000000_00_00110_0	P_NCO_FREQ	NCO frequency output
%0000_0000_000_0000000000000_00_00111_0	P_NCO_DUTY	NCO duty output
%0000_0000_000_0000000000000_00_01000_0	P_PWM_TRIANGLE	PWM triangle output
%0000_0000_000_0000000000000_00_01001_0	P_PWM_SAWTOOTH	PWM sawtooth output
%0000_0000_000_0000000000000_00_01010_0	P_PWM_SMPS	PWM switch-mode power supply I/O
%0000_0000_000_0000000000000_00_01011_0	P_QUADRATURE	A-B quadrature encoder input
%0000_0000_000_0000000000000_00_01100_0	P_REG_UP	Inc on A-rise when B-high
%0000_0000_000_0000000000000_00_01101_0	P_REG_UP_DOWN	Inc on A-rise when B-high, dec on A-rise when B-low
%0000_0000_000_0000000000000_00_01110_0	P_COUNT_RISES	Inc on A-rise, optionally dec on B-rise
%0000_0000_000_0000000000000_00_01111_0	P_COUNT_HIGHS	Inc on A-high, optionally dec on B-high
%0000_0000_000_0000000000000_00_10000_0	P_STATE_TICKS	For A-low and A-high states, count ticks
%0000_0000_000_0000000000000_00_10001_0	P_HIGH_TICKS	For A-high states, count ticks
%0000_0000_000_0000000000000_00_10010_0	P_EVENTS_TICKS	For X A-highs/rises/edges, count ticks / Timeout on X ticks of no A-high/rise/edge
%0000_0000_000_0000000000000_00_10011_0	P_PERIODS_TICKS	For X periods of A, count ticks
%0000_0000_000_0000000000000_00_10100_0	P_PERIODS_HIGHS	For X periods of A, count highs
%0000_0000_000_0000000000000_00_10101_0	P_COUNTER_TICKS	For periods of A in X+ ticks, count ticks
%0000_0000_000_0000000000000_00_10110_0	P_COUNTER_HIGHS	For periods of A in X+ ticks, count highs
%0000_0000_000_0000000000000_00_10111_0	P_COUNTER_PERIODS	For periods of A in X+ ticks, count periods
%0000_0000_000_0000000000000_00_11000_0	P_ADC	ADC sample/filter/capture, internally clocked
%0000_0000_000_0000000000000_00_11001_0	P_ADC_EXT	ADC sample/filter/capture, externally clocked
%0000_0000_000_0000000000000_00_11010_0	P_ADC_SCOPE	ADC scope with trigger
%0000_0000_000_0000000000000_00_11011_0	P_USB_PAIR	USB pin pair
%0000_0000_000_0000000000000_00_11100_0	P_SYNC_TX	Synchronous serial transmit
%0000_0000_000_0000000000000_00_11101_0	P_SYNC_RX	Synchronous serial receive
%0000_0000_000_0000000000000_00_11110_0	P_ASYNC_TX	Asynchronous serial transmit
%0000_0000_000_0000000000000_00_11111_0	P_ASYNC_RX	Asynchronous serial receive

Built-In Symbols for Streamer Modes

Streamer Symbol Value	Symbol Name
#Immediate → LUT → Pins / DACs	
%0000_0000_0000_0000 << 16 %0000_DDDD_EPPP_BBBB << 16	X_IMM_32X1_LUT
%0001_0000_0000_0000 << 16 %0001_DDDD_EPPP_BBBB << 16	X_IMM_16X2_LUT
%0010_0000_0000_0000 << 16 %0010_DDDD_EPPP_BBBB << 16	X_IMM_8X4_LUT
%0011_0000_0000_0000 << 16 %0011_DDDD_EPPP_BBBB << 16	X_IMM_4X8_LUT
#Immediate → Pins / DACs	
%0100_0000_0000_0000 << 16 %0100_DDDD_EPPP_PPPA << 16	X_IMM_32X1_1DAC1
%0101_0000_0000_0000 << 16 %0101_DDDD_EPPP_PP0A << 16	X_IMM_16X2_2DAC1
%0101_0000_0000_0010 << 16 %0101_DDDD_EPPP_PP1A << 16	X_IMM_16X2_1DAC2
%0110_0000_0000_0000 << 16 %0110_DDDD_EPPP_P00A << 16	X_IMM_8X4_4DAC1
%0110_0000_0000_0010 << 16 %0110_DDDD_EPPP_P01A << 16	X_IMM_8X4_2DAC2
%0110_0000_0000_0100 << 16 %0110_DDDD_EPPP_P10A << 16	X_IMM_8X4_1DAC4
%0110_0000_0000_0110 << 16 %0110_DDDD_EPPP_0110 << 16	X_IMM_4X8_4DAC2
%0110_0000_0000_0111 << 16 %0110_DDDD_EPPP_0111 << 16	X_IMM_4X8_2DAC4
%0110_0000_0000_1110 << 16 %0110_DDDD_EPPP_1110 << 16	X_IMM_4X8_1DAC8
%0110_0000_0000_1111 << 16 %0110_DDDD_EPPP_1111 << 16	X_IMM_2X16_4DAC4
%0111_0000_0000_0000 << 16 %0111_DDDD_EPPP_0000 << 16	X_IMM_2X16_2DAC8
%0111_0000_0000_0001 << 16 %0111_DDDD_EPPP_0001 << 16	X_IMM_1X32_4DAC8
RDFAST → LUT → Pins / DACs	
%0111_0000_0000_0010 << 16 %0111_DDDD_EPPP_001A << 16	X_RFLONG_32X1_LUT
%0111_0000_0000_0100 << 16 %0111_DDDD_EPPP_010A << 16	X_RFLONG_16X2_LUT
%0111_0000_0000_0110 << 16 %0111_DDDD_EPPP_011A << 16	X_RFLONG_8X4_LUT
%0111_0000_0000_1000 << 16 %0111_DDDD_EPPP_1000 << 16	X_RFLONG_4X8_LUT
RDFAST → Pins / DACs	
%1000_0000_0000_0000 << 16 %1000_DDDD_EPPP_PPPA << 16	X_RFBYTE_1P_1DAC1
%1001_0000_0000_0000 << 16 %1001_DDDD_EPPP_PP0A << 16	X_RFBYTE_2P_2DAC1
%1001_0000_0000_0010 << 16 %1001_DDDD_EPPP_PP1A << 16	X_RFBYTE_2P_1DAC2
%1010_0000_0000_0000 << 16 %1010_DDDD_EPPP_P00A << 16	X_RFBYTE_4P_4DAC1
%1010_0000_0000_0010 << 16 %1010_DDDD_EPPP_P01A << 16	X_RFBYTE_4P_2DAC2
%1010_0000_0000_0100 << 16 %1010_DDDD_EPPP_P10A << 16	X_RFBYTE_4P_1DAC4
%1010_0000_0000_0110 << 16 %1010_DDDD_EPPP_0110 << 16	X_RFBYTE_8P_4DAC2
%1010_0000_0000_0111 << 16 %1010_DDDD_EPPP_0111 << 16	X_RFBYTE_8P_2DAC4
%1010_0000_0000_1110 << 16 %1010_DDDD_EPPP_1110 << 16	X_RFBYTE_8P_1DAC8
%1010_0000_0000_1111 << 16	X_RFWORD_16P_4DAC4